Clínica PSP

Agustín Borreguero Castro

Índice

- Introducción
- Desarrollo y funcionalidades
- Gestión de concurrencia
- Comunicación en red
- Base de datos
- Desafíos y aprendizajes
- Futuras Mejoras

Introducción

En esta presentación se expone el desarrollo de un sistema de gestión de procesos para una clínica veterinaria. El objetivo principal del proyecto es simular el funcionamiento real de una clínica que dispone de dos consultas simultáneas, un sistema de urgencias y tres veterinarios.

El sistema permite gestionar la llegada de pacientes, priorizar correctamente los casos urgentes, asignar automáticamente el veterinario menos ocupado, comunicar los eventos en tiempo real mediante sockets y registrar toda la actividad en una base de datos SQLite. Con este proyecto se aplican de forma práctica los conceptos de concurrencia, sincronización, comunicación en red y persistencia de datos.

Desarrollo y funcionalidades

El sistema desarrollado permite simular el funcionamiento completo de la clínica. Cuando se registran pacientes, estos se añaden a diferentes colas según sean consultas normales o urgencias. Las urgencias tienen prioridad y se atienden antes que las consultas normales.

Las consultas están implementadas como hilos independientes, lo que permite que dos pacientes puedan ser atendidos de forma simultánea. Cada vez que una consulta solicita un paciente, el sistema selecciona automáticamente el veterinario con menor carga de trabajo.

Main.java × Atencion.java Paciente.java

```java
import db.CrearTablas;
import modelos.Paciente;
import modelos.TipoDeCaso;
import modelos.Veterinario;
import servicios.Clinica;
import servicios.Consulta;

public class Main {  new *

    public static void main(String[] args) {  new *

        // Crear tablas si no existen
        CrearTablas.crear();

        // Crear clinica y conectarla al servidor
        Clinica clinica = new Clinica();
        clinica.conectarAlServidor();

        // Añadir veterinarios
        clinica.anadirVeterinario(new Veterinario( nombre: "Veterinario_1"));
        clinica.anadirVeterinario(new Veterinario( nombre: "Veterinario_2"));
        clinica.anadirVeterinario(new Veterinario( nombre: "Veterinario_3"));

        // Registrar pacientes
        clinica.registrarPaciente(new Paciente( nombre: "Angel", TipoDeCaso.CONSULTA_NORMAL,  prioridad: 5));
        clinica.registrarPaciente(new Paciente( nombre: "Pablo", TipoDeCaso.CONSULTA_NORMAL,  prioridad: 5));
        clinica.registrarPaciente(new Paciente( nombre: "Daniel", TipoDeCaso.URGENCIA,  prioridad: 1));
        clinica.registrarPaciente(new Paciente( nombre: "Carmen", TipoDeCaso.URGENCIA,  prioridad: 2));
        clinica.registrarPaciente(new Paciente( nombre: "Sara", TipoDeCaso.CONSULTA_NORMAL,  prioridad: 5));

        // Crear consultas (hilos)
        Consulta c1 = new Consulta( nombreConsulta: "Consulta_1", clinica);
        Consulta c2 = new Consulta( nombreConsulta: "Consulta_2", clinica);


        c1.start();
        c2.start();
    }
}
```

```
[Consulta_1] Atiende a Veterinario_1 -> Paciente (Id:3', nombre:'Daniel', tipo:URGENCIA, prioridad:1, creado:2026-03-03T19:28:01.917894500Z) | Carga del veterinario:1
[Consulta_2] Atiende a Veterinario_2 -> Paciente (Id:4', nombre:'Carmen', tipo:URGENCIA, prioridad:2, creado:2026-03-03T19:28:01.924900800Z) | Carga del veterinario:1
[Consulta_1] Termina Veterinario_1 -> Paciente 3 | Carga del veterinario:0
[Consulta_2] Termina Veterinario_2 -> Paciente 4 | Carga del veterinario:0
[Consulta_2] Atiende a Veterinario_2 -> Paciente (Id:2', nombre:'Pablo', tipo:CONSULTA_NORMAL, prioridad:5, creado:2026-03-03T19:28:01.910887300Z) | Carga del veterinario:1
[Consulta_1] Atiende a Veterinario_1 -> Paciente (Id:1', nombre:'Angel', tipo:CONSULTA_NORMAL, prioridad:5, creado:2026-03-03T19:28:01.885865Z) | Carga del veterinario:1
[Consulta_2] Termina Veterinario_2 -> Paciente 2 | Carga del veterinario:0
[Consulta_2] Atiende a Veterinario_2 -> Paciente (Id:5', nombre:'Sara', tipo:CONSULTA_NORMAL, prioridad:5, creado:2026-03-03T19:28:01.930906Z) | Carga del veterinario:1
[Consulta_1] Termina Veterinario_1 -> Paciente 1 | Carga del veterinario:0
```

Desarrollo y funcionalidades

Durante el proceso de atención se registran en la base de datos tanto la hora de inicio como la hora de finalización, permitiendo llevar un seguimiento completo de cada caso. Además, cada evento importante (nuevo paciente, inicio de atención y fin de atención) se envía al servidor mediante sockets, que lo reenvía a todos los clientes conectados en tiempo real.

De esta manera, el sistema no solo gestiona procesos concurrentes, sino que también incorpora comunicación en red y persistencia de datos.

Tables | Database Metadata

Table: atenciones Page: 0 Jump <<

id	pacient...	veterina...	consulta	inicio	fin
1	3	Veterinari...	Consulta_1	2026-03...	2026-03...
2	4	Veterinari...	Consulta_2	2026-03...	2026-03...
3	2	Veterinari...	Consulta_2	2026-03...	2026-03...
4	1	Veterinari...	Consulta_1	2026-03...	2026-03...
5	5	Veterinari...	Consulta_2	2026-03...	2026-03...

Run | Servidor | Main | Cliente

```
WARNING: java.lang.System::load has been called by org.sqlite.SQLiteJDBCLoader in an unnamed module (file:/C:/Users/agubc/.m2/repository/org/xerial/sqlite-jdbc/3.49.1.0/sqlite-
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

Tablas creadas / comprobadas OK
Clínica conectada al servidor de notificaciones.
[Consulta_2] Atiende a Veterinario_2 -> Paciente (Id:4', nombre:'Carmen', tipo:URGENCIA, prioridad:2, creado:2026-03-03T19:52:01.589200Z) | Carga del veterinario:1
[Consulta_1] Atiende a Veterinario_1 -> Paciente (Id:3', nombre:'Daniel', tipo:URGENCIA, prioridad:1, creado:2026-03-03T19:52:01.584195400Z) | Carga del veterinario:1
[Consulta_2] Termina Veterinario_2 -> Paciente 4 | Carga del veterinario:0
[Consulta_1] Termina Veterinario_1 -> Paciente 3 | Carga del veterinario:0
[Consulta_2] Atiende a Veterinario_2 -> Paciente (Id:1', nombre:'Angel', tipo:CONSULTA_NORMAL, prioridad:5, creado:2026-03-03T19:52:01.563177Z) | Carga del veterinario:1
[Consulta_1] Atiende a Veterinario_1 -> Paciente (Id:2', nombre:'Pablo', tipo:CONSULTA_NORMAL, prioridad:5, creado:2026-03-03T19:52:01.578189700Z) | Carga del veterinario:1
[Consulta_2] Termina Veterinario_2 -> Paciente 1 | Carga del veterinario:0
[Consulta_1] Termina Veterinario_1 -> Paciente 2 | Carga del veterinario:0
[Consulta_2] Atiende a Veterinario_2 -> Paciente (Id:5', nombre:'Sara', tipo:CONSULTA_NORMAL, prioridad:5, creado:2026-03-03T19:52:01.595206900Z) | Carga del veterinario:1
[Consulta_2] Termina Veterinario_2 -> Paciente 5 | Carga del veterinario:0
```

idaNotaPSP > src > Main > main

Cliente

```
NOTIFICACIÓN: Conectado al sistema de la clínica
NOTIFICACIÓN: Nuevo paciente: Angel (CONSULTA_NORMAL, prioridad 5)
NOTIFICACIÓN: Nuevo paciente: Pablo (CONSULTA_NORMAL, prioridad 5)
NOTIFICACIÓN: Nuevo paciente: Daniel (URGENCIA, prioridad 1)
NOTIFICACIÓN: Nuevo paciente: Carmen (URGENCIA, prioridad 2)
NOTIFICACIÓN: Nuevo paciente: Sara (CONSULTA_NORMAL, prioridad 5)
NOTIFICACIÓN: INICIO -> Consulta_2: Veterinario_2 atiende a Carmen
NOTIFICACIÓN: INICIO -> Consulta_1: Veterinario_1 atiende a Daniel
NOTIFICACIÓN: FIN -> Consulta_2: Veterinario_2 terminó con Carmen
NOTIFICACIÓN: FIN -> Consulta_1: Veterinario_1 terminó con Daniel
NOTIFICACIÓN: INICIO -> Consulta_2: Veterinario_2 atiende a Angel
NOTIFICACIÓN: INICIO -> Consulta_1: Veterinario_1 atiende a Pablo
NOTIFICACIÓN: FIN -> Consulta_2: Veterinario_2 terminó con Angel
NOTIFICACIÓN: FIN -> Consulta_1: Veterinario_1 terminó con Pablo
NOTIFICACIÓN: INICIO -> Consulta_2: Veterinario_2 atiende a Sara
NOTIFICACIÓN: FIN -> Consulta_2: Veterinario_2 terminó con Sara
```

Gestión de concurrencia

El sistema utiliza programación concurrente mediante hilos.

Cada consulta se ejecuta como un hilo independiente:

- Consulta_1
- Consulta_2

Esto permite atender pacientes de forma simultánea.

Para evitar errores se utilizan mecanismos de sincronización como:

- synchronized
- wait()
- notifyAll()

De esta forma se controla el acceso a las colas de pacientes.

Comunicación en red

El sistema implementa comunicación cliente-servidor mediante sockets.

Componentes:

1. **Servidor**
   - Escucha conexiones
   - Recibe mensajes
   - Los envía a todos los clientes
2. **Clientes**
   - Reciben notificaciones en tiempo real

Esto permite que diferentes clientes vean lo que ocurre en la clínica.

Base de datos

Para almacenar la información del sistema se utiliza SQLite, una base de datos ligera que funciona directamente dentro de la aplicación sin necesidad de instalar un servidor externo.
La conexión con la base de datos se realiza mediante JDBC, que permite ejecutar consultas SQL desde el programa en Java.

En el proyecto se utilizan principalmente dos tablas:

Tabla pacientes
Esta tabla almacena la información básica de cada paciente que entra en la clínica.

- id → identificador único del paciente
- nombre → nombre del paciente
- tipo → indica si es una consulta normal o una urgencia
- prioridad → nivel de urgencia del caso

id	nombre	tipo	prioridad
1	Angel	CONSULTA_NORMAL	5
2	Pablo	CONSULTA_NORMAL	5
3	Daniel	URGENCIA	1
4	Carmen	URGENCIA	2
5	Sara	CONSULTA_NORMAL	5

Tabla atenciones
Esta tabla registra toda la información relacionada con las atenciones realizadas por los veterinarios.

- id → identificador de la atención
- paciente_id → paciente que está siendo atendido
- veterinario → veterinario asignado
- consulta → consulta donde se realiza la atención
- inicio → hora en la que comienza la atención
- fin → hora en la que termina la atención

id	pacient...	veterinario	consulta	inicio	fin
1	4	Veterinario_2	Consulta_2	2026-03-03...	2026-03...
2	3	Veterinario_1	Consulta_1	2026-03-03...	2026-03...
3	1	Veterinario_2	Consulta_2	2026-03-03...	2026-03...
4	2	Veterinario_1	Consulta_1	2026-03-03...	2026-03...
5	5	Veterinario_2	Consulta_2	2026-03-03...	2026-03...

Desafíos y aprendizajes

Durante el desarrollo del proyecto surgieron varios desafíos técnicos. Uno de los principales fue la gestión correcta de la concurrencia, ya que al trabajar con varios hilos era necesario evitar accesos simultáneos a las listas compartidas. Este problema se solucionó aplicando correctamente la sincronización en los métodos críticos.

Otro problema fue la conexión entre cliente y servidor mediante sockets. Al principio los mensajes no se enviaban correctamente porque el servidor no reenviaba los mensajes recibidos. Tras revisar la arquitectura, se implementó un sistema de broadcast que permitió enviar notificaciones a todos los clientes conectados.

También se detectaron errores relacionados con claves primarias duplicadas en la base de datos al ejecutar el programa varias veces.

Futuras Mejoras

El sistema podría mejorarse en futuras versiones incorporando una interfaz gráfica para facilitar la interacción del usuario.

Otra mejora posible sería implementar un sistema de autenticación para diferenciar tipos de usuarios.

Finalmente, se podría ampliar la base de datos para almacenar un historial más completo de cada paciente.

Gracias